

Flutter Widget Documentation:

CustomStepIndicator

Overview

`CustomStepIndicator` is a reusable Flutter widget that visually displays progress through multiple steps in a process (for example: form steps, booking flow, onboarding, etc.).

It shows:

- A horizontal progress line
- Circular step indicators
- Step labels
- Completed step check icons

This pattern is commonly used in:

- Multi-step forms
- Checkout flows
- Appointment booking
- Onboarding processes

Widget Constructor

```
const CustomStepIndicator({
  super.key,
  required this.currentStep,
  required this.steps,
});
```

Parameters

Parameter	Type	Description
currentStep	int	The currently active step
steps	List<String>	Labels for each step

Example Usage

```
CustomStepIndicator(  
  currentStep: 2,  
  steps: [  
    "Info",  
    "Details",  
    "Payment",  
    "Complete",  
  ],  
)
```

Result:

- Step 1 → Completed
- Step 2 → Active
- Step 3 → Pending
- Step 4 → Pending

Layout Structure

The widget layout is built using a `Stack`.

```
Stack  
├─ Background line  
├─ Progress line  
└─ Steps row (circle + label)
```

This allows the progress line to appear behind the circles.

Step-by-Step Logic

1 Background Line

```
Positioned(  
  top: 16.h,  
  left: 20.w,  
  right: 20.w,  
  child: Container(  
    width: 20.w,  
    height: 16.h,  
    color: Colors.grey,  ),  
)
```

```

        height: 2,
        color: AppColors.cE5E7EB,
    ),
)

```

Purpose:

Creates the full grey line across all steps.

2 Progress Line

```

Row(
  children: List.generate(steps.length - 1, (index) {
    return Expanded(
      child: Container(
        height: 2,
        color: index < currentStep - 1
          ? AppColors.c0D6EFD
          : Colors.transparent,
      ),
    );
  }),
)

```

Logic:

```

If step is completed
→ show blue progress line

Else
→ keep transparent

```

Example:

Steps = 4 CurrentStep = 3

Visible progress:

```

Step1 — Step2 — Step3 — Step4
  ✓           ✓           •           ○

```

3 Generate Steps

```
List.generate(steps.length, (index) {
```

This dynamically creates all steps.

Each step calculates:

```
stepNum = index + 1
```

4 Step State Detection

```
final isActive = stepNum == currentStep;  
final isCompleted = stepNum < currentStep;
```

Possible states:

State	Condition
Completed	stepNum < currentStep
Active	stepNum == currentStep
Pending	stepNum > currentStep

Circle UI

```
Container(  
  width: 32.w,  
  height: 32.w,  
  decoration: BoxDecoration(  
    shape: BoxShape.circle,  
    color: isActive || isCompleted  
      ? AppColors.c0D6EFD  
      : const Color(0xFF3F4F6),  
  ),  
)
```

Meaning:

Step State	Circle Color
Completed	Blue
Active	Blue
Pending	Light grey

Completed Icon

```
isCompleted  
? Icon(Icons.check_rounded)
```

If the step is completed:

- A check icon appears

Otherwise:

- The step number is displayed

Step Number

```
Text('$stepNum')
```

Displayed only when:

```
step not completed
```

Step Label

```
Text(  
  steps[index],  
)
```

This shows the label below the step circle.

Example labels:

Info
Details
Payment
Complete

Color Logic for Labels

```
color: stepNum <= currentStep  
      ? AppColors.c0D6EFD  
      : AppColors.c9CA3AF
```

Meaning:

Step	Label Color
Completed	Blue
Active	Blue
Pending	Grey

Visual Example

✓ ✓ • ○
Info Details Payment Complete

Legend:

✓ Completed
• Active
○ Pending

Advantages

- Fully reusable
 - Dynamic step count
 - Supports any label
 - Clean UI
 - Works with responsive sizing
-

Possible Improvements

You can extend this widget with:

- Tapable steps
 - Animated progress line
 - Step icons instead of numbers
 - Vertical step indicator
 - Gradient progress line
-

Full Source Code

```
import 'package:thedoctor0101/constants/common_imports.dart';

class CustomStepIndicator extends StatelessWidget {
  final int currentStep;
  final List<String> steps;

  const CustomStepIndicator({
    super.key,
    required this.currentStep,
    required this.steps,
  });

  @override
  Widget build(BuildContext context) {
    return SizedBox(
      height: 80.h,
      child: Stack(
        children: [
          Positioned(
            top: 16.h,
            left: 20.w,
```

```

        right: 20.w,
        child: Container(
          height: 2,
          color: AppColors.cE5E7EB,
        ),
      ),
    ),

    Positioned(
      top: 16.h,
      left: 20.w,
      right: 20.w,
      child: Row(
        children: List.generate(steps.length - 1, (index) {
          return Expanded(
            child: Container(
              height: 2,
              color: index < currentStep - 1
                ? AppColors.c0D6EFD
                : Colors.transparent,
            ),
          );
        })),
      ),
    ),

    Row(
      mainAxisAlignment: MainAxisAlignment.spaceBetween,
      children: List.generate(steps.length, (index) {
        final stepNum = index + 1;
        final isActive = stepNum == currentStep;
        final isCompleted = stepNum < currentStep;

        return Column(
          mainAxisAlignment: MainAxisAlignment.min,
          children: [
            Container(
              width: 32.w,
              height: 32.w,
              decoration: BoxDecoration(
                shape: BoxShape.circle,
                color: isActive || isCompleted
                  ? AppColors.c0D6EFD
                  : const Color(0xFFFF3F4F6),
              ),
            ),
            child: Center(
              child: isCompleted
                ? Icon(
                    Icons.check_rounded,

```


